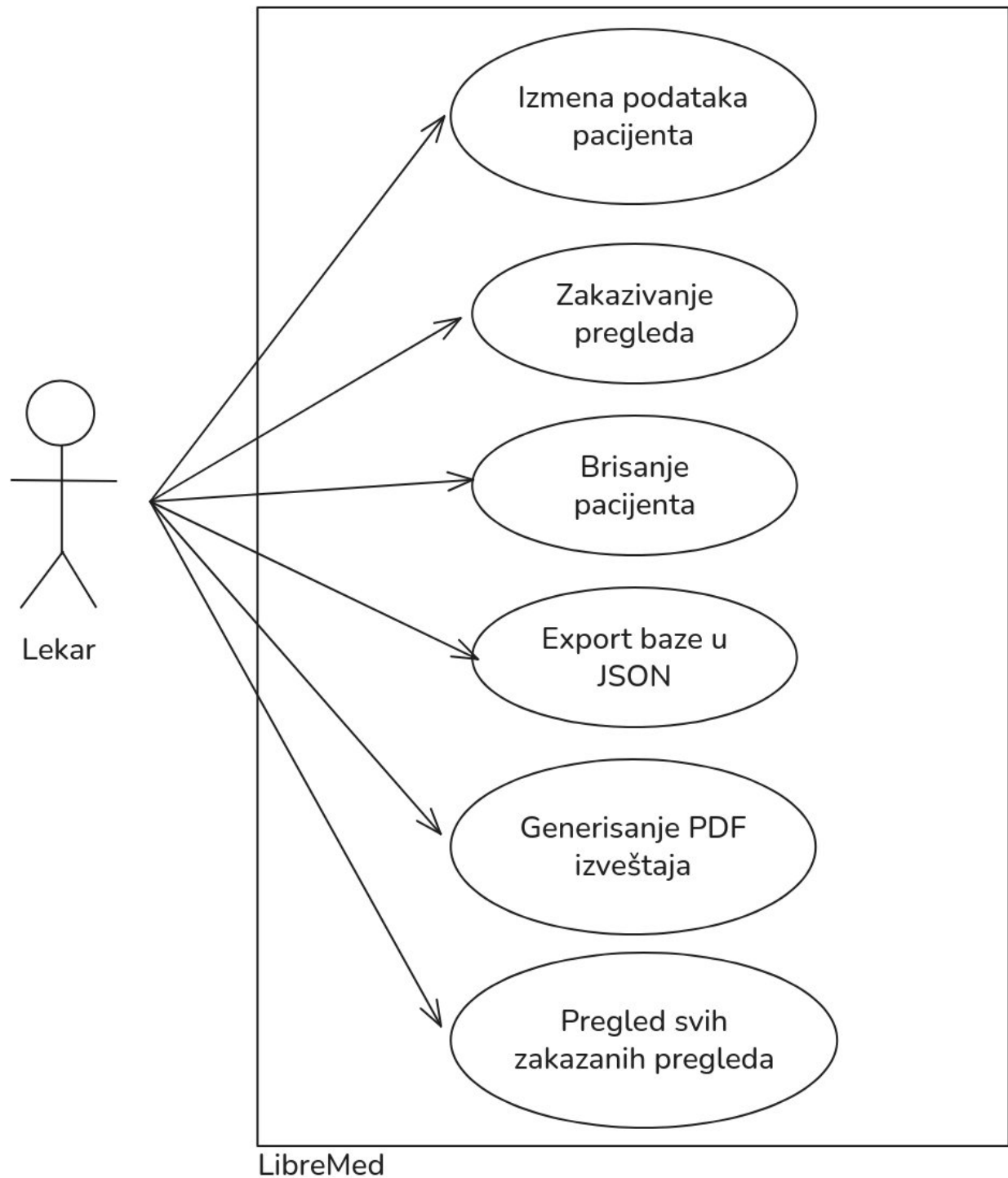


Slučajevi upotrebe

Dijagram slučajeva upotrebe



Opis slučajeva upotrebe

Izmena podataka o pacijentu

Preduslov	Pacijent postoji
Opis	Lekar otvara profil željenog pacijenta, menja proizvoljne podatke o njemu, i snima izmene u bazu podataka.

Zakazivanje pregleda

Preduslov	Pacijent postoji
Opis	Lekar otvara profil pacijenta, otvara ekran sa njegovim pregledima, i unosi podatke o novom pregledu.

Brisanje pacijenta

Preduslov	Pacijent postoji
Opis	Lekar otvara profil pacijenta, i pritiska dugme za brisanje njegovog profila.

Export baze u JSON

Preduslov	n/a
Opis	Lekar otvara glavni ekran programa, i pritiska dugme za eksport baze u JSON. Bira lokaciju gde će se sačuvati datoteka, kao i njeno ime.

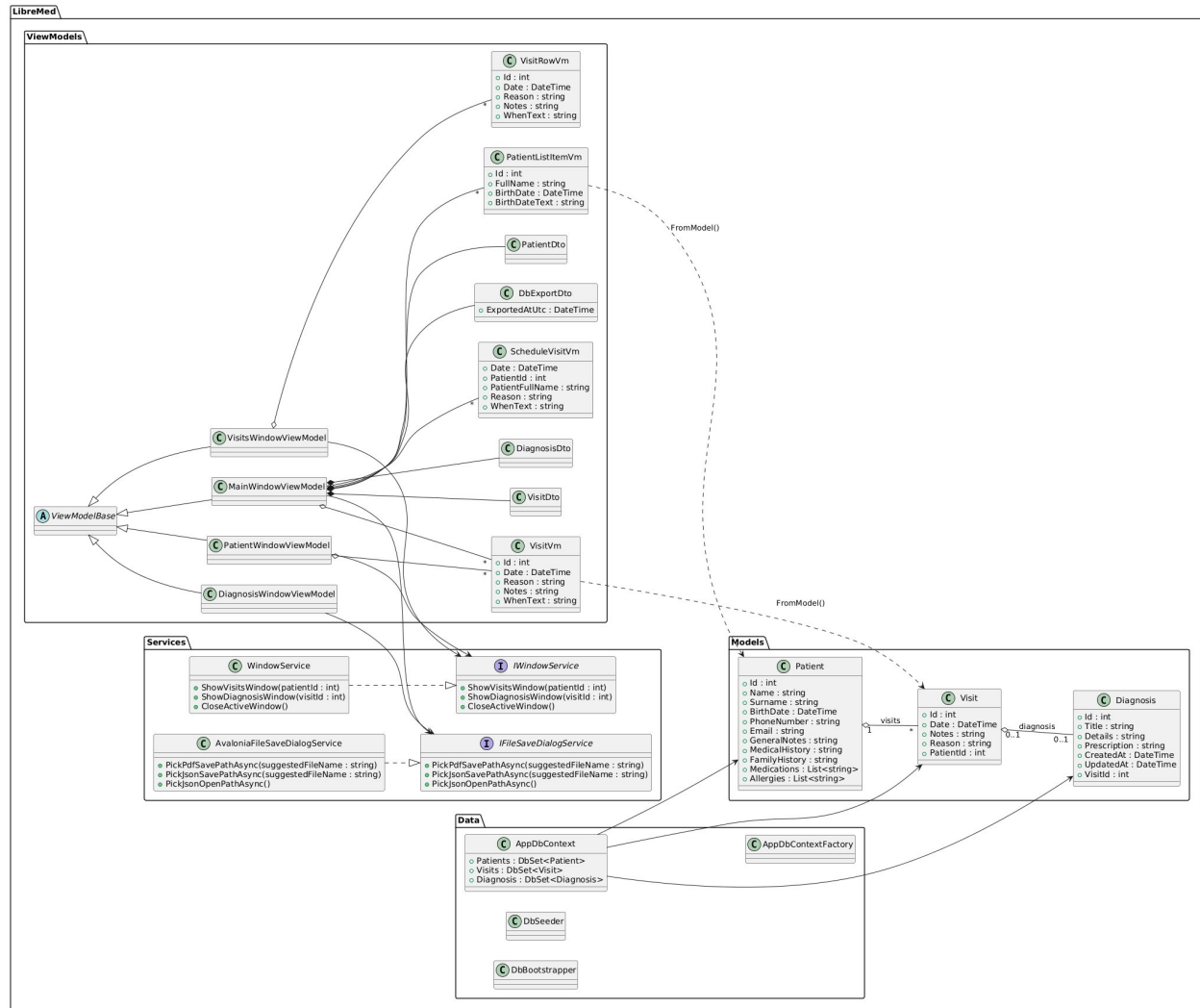
Generisanje PDF izveštaja

Preduslov	Dijagnoza postoji
Opis	Lekar bira dijagnozu željenog pregleda datog pacijenta, i pritiska dugme za generisanje PDF izveštaja. Bira lokaciju gde će se sačuvati datoteka, i menja njeno ime ako mu se ne dopada preporučeno ime.

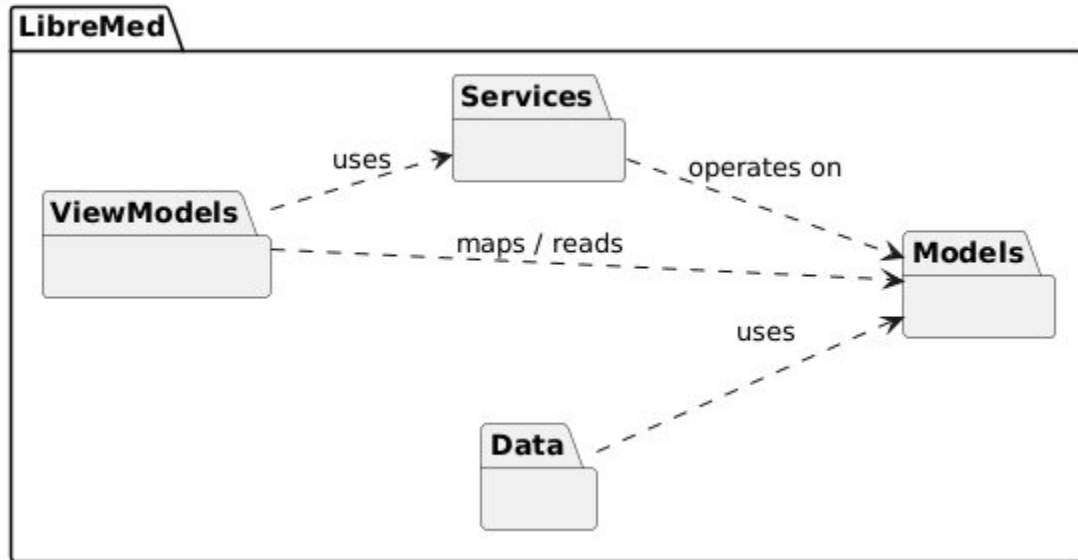
Pregled svih zakazanih pregleda

Preduslov	n/a
Opis	Lekar pritiska <i>Schedule</i> dugme na vrhu glavnog prozora.

Dijagram klasa

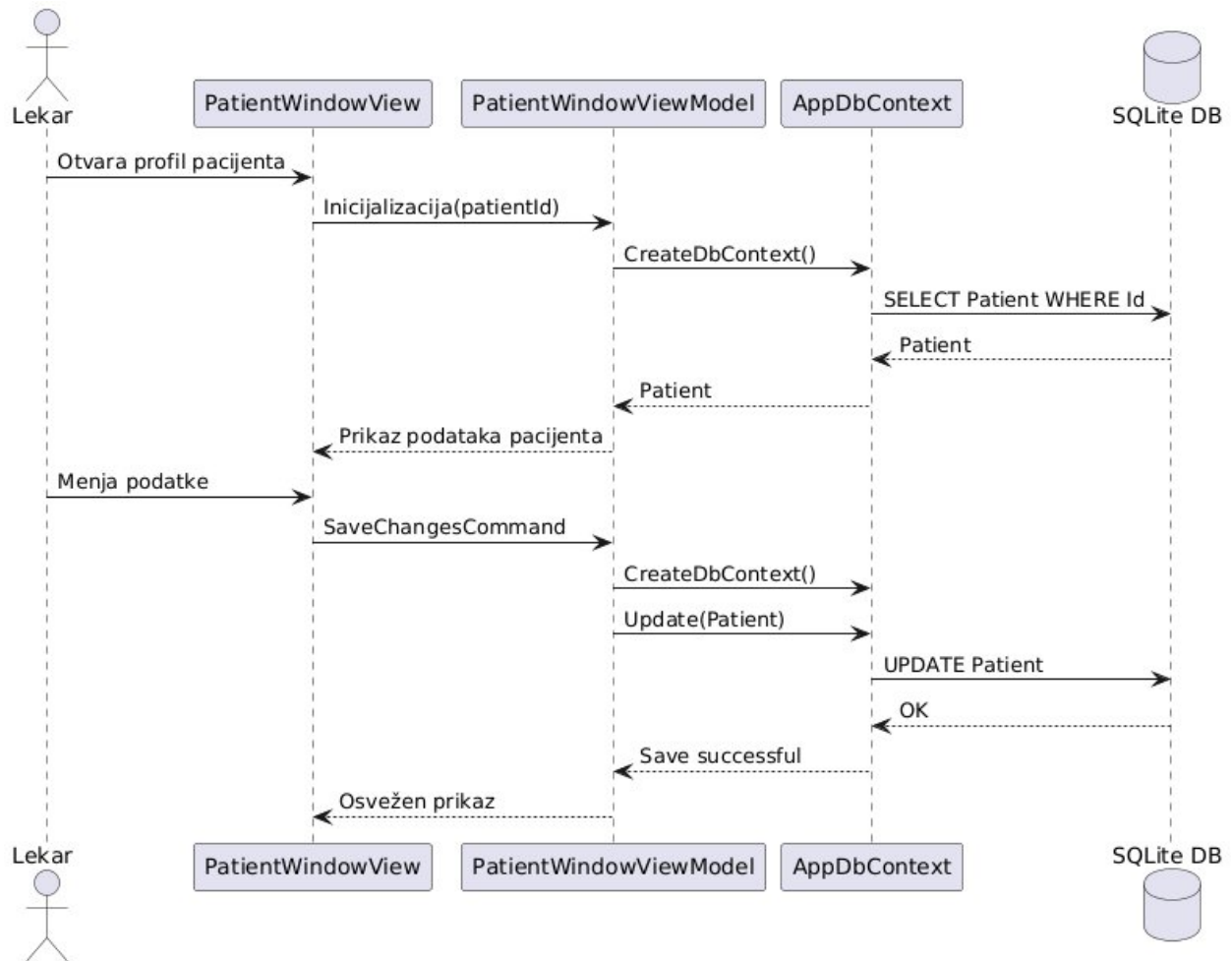


Dijagram paketa

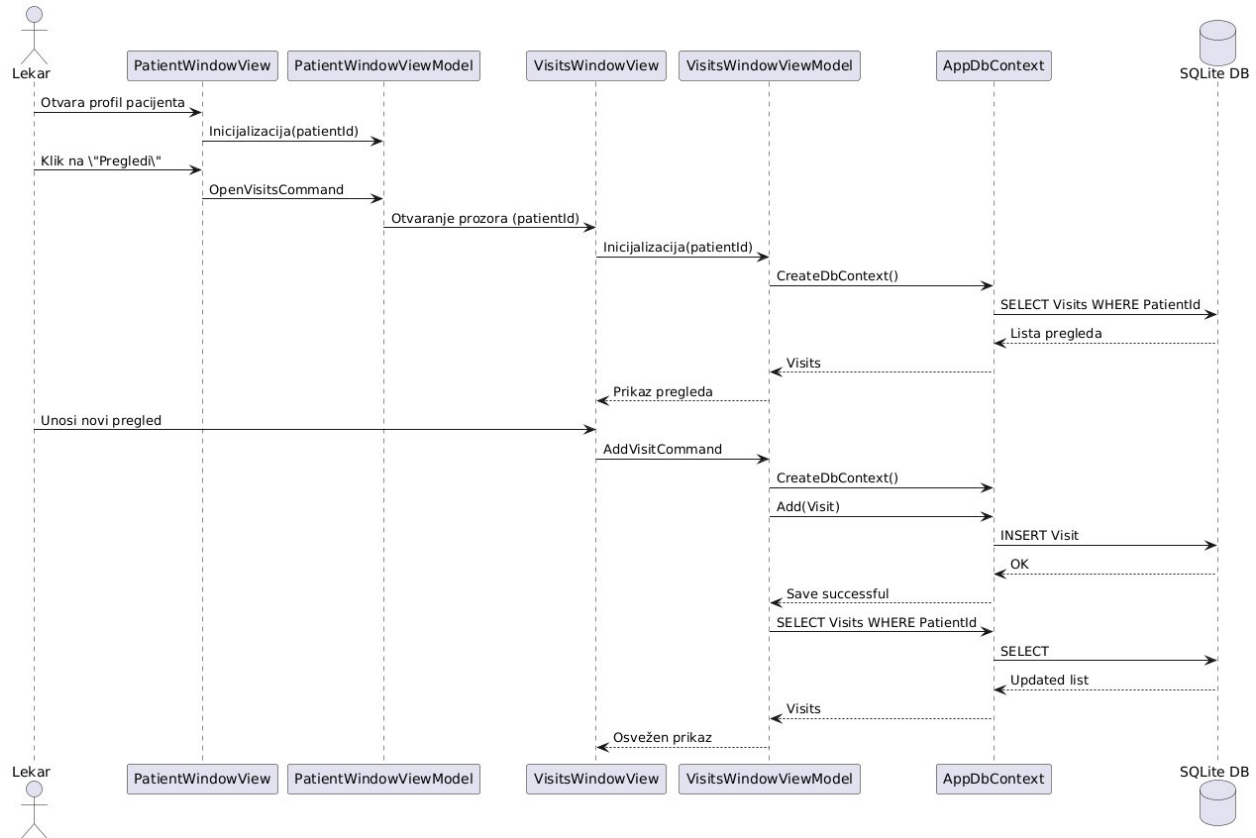


Dijagrami sekvenci

Izmena podataka o pacijentu



Zakazivanje pregleda



Arhitektura projekta

Folder struktura (šta gde živi)

LibreMed/	Glavni projekat (Avalonia desktop app).
LibreMed/Views/	UI sloj – XAML fajlovi (*.axaml) i njihovi code-behind fajlovi (*.axaml.cs). Ovde su definisani prozori, layout, kontrole, binding-i ka ViewModel-ima. Tipično: MainWindow, PatientWindow, VisitsWindow, DiagnosisWindow.
LibreMed/ViewModels/	MVVM “mozak” aplikacije – ViewModel klase koje drže stanje (properties) i logiku UI akcija (commands). ViewModels koriste servise i Data sloj da učitaju/sačuvaju podatke, a View samo bind-uje na njih.
LibreMed/Models/	Domen/entiteti – “čisti” modeli koji predstavljaju podatke u bazi i poslovnom domenu (Patient, Visit, Diagnosis). Ovo su klase koje EF Core mapira na tabele (ili strukture u SQLite).

LibreMed/Data/	Pristup podacima (EF Core + SQLite): AppDbContext (DbSet-ovi, konfiguracija relacija, konverzije npr. liste u JSON) bootstrap/seed/factory klase za kreiranje baze i inicijalne podatke
LibreMed/Services/	Servisi koji apstrahuju platformu/UI funktionalnosti od ViewModel-a: npr. servis za otvaranje novih prozora (IWindowService, WindowService) servis za dijaloge/čuvanje fajlova (IFileSaveDialogService, implementacija na Avalonia storage provider)
Root fajlovi (Program.cs, App.axaml, App.axaml.cs, ViewLocator.cs)	“Entry” i kompozicija aplikacije: podizanje Avalonia aplikacije, registracija servisa, resursi/teme, mapiranje VM → View (ViewLocator).
LibreMed/Migrations/	EF Core migracije (automatski generisano). Bitno za evoluciju šeme baze, ali nije “core” arhitekture aplikacije.

Primena dizajn šablona

Factory pattern (kreiranje objekata)

AppDbContextFactory fabrika standardizuje kako se pravi (connection string, opcije...).

AppDbContextFactoryAppDbContext.

Service pattern + Dependency Inversion (apstrakcije kroz interfejs)

Unutar Services/ (IWindowService, IFileSaveDialogService) + implementacije.
Kako: ViewModel zavisi od interfejsa, ne od konkretne Avalonia implementacije.
Primer:

- IWindowService sakriva detalje kreiranja/otvaranja prozora
- IFileSaveDialogService sakriva detalje file picker-a